# บทที่ 3 — ใส่มาทำไม ตรงกับงานเราไหม

เอกสารนี้ไม่ใช่ส่วนของรูปเล่ม ไล่อธิบายทุกหัวข้อของบทที่ 3 ว่า **ใส่มาทำไม · ตัวอย่างให้เห็นภาพ · ตรงกับงานเราไหม**

บทที่ 2 เป็น **ความรู้** ว่าปัญหาแบบนี้โลกเขาแก้กันอย่างไร ส่วนบทที่ 3 เป็น **การตัดสินใจ** ว่าเราเลือกทำแบบไหนและทำไม ถ้าถูกถามว่าสองบทต่างกันตรงไหน ตอบประโยคเดียวว่า

> บทที่ 2 บอกว่า “มีวิธีอะไรบ้าง” ส่วนบทที่ 3 บอกว่า “ผมเลือกวิธีไหน และเลือกเพราะอะไร”

ทุกหัวข้อในบทที่ 3 ต้องสืบย้อนกลับไปหาบทที่ 1 หรือบทที่ 2 ได้เสมอ ถ้าหัวข้อไหนสืบย้อนไม่ได้ แปลว่าไม่ควรมี

# ลำดับของบทนี้ คือสิ่งที่ต้องอธิบายให้ได้ก่อนอย่างอื่น

บทที่ 3 ไม่ได้เรียงตามความสะดวก แต่เรียงตาม **เส้นทางที่ข้อมูลเดินจริง**

> ปัญหา → ความต้องการ → กรณีการใช้งาน → **โมดูล** → สถาปัตยกรรม → ฐานข้อมูล → พฤติกรรม → หน้าจอ → ทดสอบ

เหตุผลที่ต้องเรียงแบบนี้คือ **แต่ละขั้นเป็นวัตถุดิบของขั้นถัดไป** เราแบ่งโมดูลไม่ได้ถ้ายังไม่รู้ว่ามีกรณีการใช้งานอะไรบ้าง และวาดสถาปัตยกรรมไม่ได้ถ้ายังไม่รู้ว่ามีโมดูลอะไร

**ประโยคที่ควรพูด** — “ผมไม่ได้เริ่มจากวาดสถาปัตยกรรมก่อนครับ ผมเริ่มจากปัญหาที่เก็บมา แปลงเป็นความต้องการ แล้วเป็น use case พอได้ use case ครบผมถึงรู้ว่าต้องมีกี่โมดูล สถาปัตยกรรมเป็นผลลัพธ์ข้อสุดท้าย ไม่ใช่ข้อแรก”

| หัวข้อ | ตอบคำถามว่า |
|---|---|
| **3.1 กระบวนการดำเนินงาน** | ทำงานเป็นระยะยังไง |
| **3.2 การวิเคราะห์ความต้องการ** | ปัญหาในบทที่ 1 กลายเป็นข้อกำหนดที่ตรวจได้อย่างไร |
| **3.3 การออกแบบกรณีการใช้งาน** | ใครทำอะไรกับระบบบ้าง |
| **3.4 การแบ่งโมดูล** | **ระบบมีกี่โมดูล และใครใช้โมดูลไหน** |
| **3.5 ภาพรวมสถาปัตยกรรม** | ระบบวางตัวอย่างไร แบ่งเป็นกี่หน่วยติดตั้ง |
| **3.6 ส่วนประกอบภายใน** | โมดูลเรียกหากันอย่างไร |
| **3.7 การออกแบบฐานข้อมูล** | เก็บข้อมูลอะไร โครงสร้างแบบไหน |
| **3.8 แผนภาพลำดับ กิจกรรม สถานะ** | ระบบทำงานอย่างไรเมื่อเวลาผ่านไป |
| **3.9 การออกแบบหน้าจอ** | ผู้ใช้เห็นอะไร |
| **3.10 แผนการทดสอบ** | จะพิสูจน์อย่างไรว่าทำสำเร็จ |

# 3.1 กระบวนการดำเนินงานโครงงาน

**ใส่มาทำไม** — คู่มือรูปเล่มกำหนดให้บทที่ 3 บอกว่าทำงานด้วยกระบวนการอะไร และอาจารย์จะถามว่าทำไมเลือกแบบนี้

**ตัวอย่างให้เห็นภาพ** — เราเลือกแบบน้ำตกที่ทวนกลับได้ (Iterative Waterfall) ไม่ใช่ Agile เพราะขอบเขตนิ่งตั้งแต่ต้นและผู้ใช้จริงมีแค่สามคน **ไม่มีลูกค้าคอยเปลี่ยนความต้องการทุกสองสัปดาห์** จึงไม่คุ้มที่จะใช้กระบวนการที่ออกแบบมาเพื่อรับการเปลี่ยนแปลงบ่อย · แต่ยังเปิดให้ย้อนกลับได้ เช่น ตอนพัฒนาระยะที่ 4 พบว่าตารางฐานข้อมูลขาดช่องเก็บเวลาที่จ่ายของ ก็ย้อนกลับไปแก้แบบในระยะที่ 2

**ตรงกับงานเราไหม** — ตรง และจุดที่ต้องชี้เองคือ **ทำไมแบ่งการพัฒนาเป็นสองระยะ (ระยะที่ 3 กับ 4)** คำตอบคือระยะที่ 4 ต้องใช้ข้อมูลคำสั่งซื้อจริงที่เกิดจากระยะที่ 3 ถ้าทำพร้อมกันจะทดสอบการยืนยันจ่ายสินค้าด้วยข้อมูลจริงไม่ได้เลย

# 3.2 การวิเคราะห์ความต้องการ

**ใส่มาทำไม** — บทที่ 1 เขียนปัญหาไว้เป็นความเรียง ซึ่ง **เอาไปทดสอบไม่ได้** ว่าแก้สำเร็จหรือยัง หัวข้อนี้จึงแปลงความเรียงให้เป็นรายการที่มีรหัสกำกับ

**ตัวอย่างให้เห็นภาพ** — บทที่ 1 เขียนว่า “ยอดสต็อกคลาดเคลื่อนจากความเป็นจริง” ประโยคนี้จะตรวจว่าผ่านหรือไม่ผ่านอย่างไร วัดไม่ได้ แต่พอแปลงเป็น

> **EV-07** ยอดสต็อกในบันทึกคลาดเคลื่อน → **FR-06** ตัดยอดในธุรกรรมเดียวกับการเปลี่ยนสถานะ และป้องกันการจ่ายซ้ำ → **AC-08** ถ้าสต็อกไม่พอ ธุรกรรมต้องถูกยกเลิกทั้งหมด

ตอนนี้เอาไปทำเป็นกรณีทดสอบได้ทันที

**ตรงกับงานเราไหม** — ตรง และตัวเลขทุกตัวมีที่มา เช่น **NFR-02 เกณฑ์ 3 วินาที** ไม่ได้ตั้งลอย ๆ มาจากการประมาณว่าลูกค้าเดินจากจุดชำระเงินไปคลังใช้เวลาราวหนึ่งนาที ผลักข้อมูลถึงในไม่กี่วินาทีจึงพอให้คลังเริ่มจัดของก่อน · ส่วน **NFR-06 สิบเซสชัน** มาจากจำนวนพนักงานจริงของร้าน

**จุดที่ต้องแยกให้ออก** — FR กับ BR ต่างกัน **BR คือกฎที่เป็นจริงอยู่แล้วแม้ไม่มีคอมพิวเตอร์** เช่น “ตัวเลขในระบบต้องตรงของจริง” ร้านใช้กฎนี้มาตั้งแต่จดใส่สมุด ส่วน FR คือสิ่งที่ระบบต้องทำเพื่อให้กฎนั้นเป็นจริง

# 3.3 การออกแบบกรณีการใช้งาน

**ใส่มาทำไม** — เป็นสะพานระหว่างความต้องการกับโมดูล อาจารย์บอกไว้ตรง ๆ ว่า **use case คือสิ่งที่บอกว่าระบบมีกี่โมดูลและแต่ละโมดูลใครใช้**

**ตัวอย่างให้เห็นภาพ** — ทำไม **UC-04 สร้างเลขที่คำสั่งซื้อไม่ซ้ำ** ถึงแยกออกมาเป็นรหัสของตัวเอง ทั้งที่ไม่มีผู้ใช้คนไหนกดมัน

> เพราะมันมีเงื่อนไขความถูกต้องของตัวเองที่ต้องทดสอบแยก สมมติพนักงานขายสองคนกดออกใบเสร็จพร้อมกันตอน 9 โมงเช้า ทั้งคู่จะขอเลขลำดับถัดไปตัวเดียวกัน **ถ้าไม่แยกออกมา จะไม่มีที่ให้เขียนว่าระบบ**

> **ต้องจัดการกรณีนี้อย่างไร**

**ตรงกับงานเราไหม** — ตรง และในตารางที่ 3.9 กับ 3.10 มีศัพท์สองคำที่คนสับสนบ่อยที่สุด

| คำ | แปลว่า | ตัวอย่างในงานเรา |
|---|---|---|
| **Alternate Flow** | ไปคนละทางแต่**ยังถึงที่หมาย** | WebSocket ต่อไม่ได้ ระบบสลับไป SSE งานยังเดินต่อ |
| **Exception Flow** | **ไปไม่ถึง** จบด้วยความล้มเหลว | สต็อกไม่พอ ระบบย้อนธุรกรรมทั้งหมด |

# 3.4 การแบ่งโมดูล — หัวข้อที่อาจารย์ถามตรง ๆ

หัวข้อนี้สำคัญที่สุดในรอบนี้ จึงแยกอธิบายละเอียดไว้ในหน้าถัดไป

**ใส่มาทำไม** — ตอบคำถามของอาจารย์ว่า **ระบบเรามีกี่โมดูล และแต่ละโมดูลใครเป็นคนใช้** ซึ่งบทที่ 3 ฉบับเดิมไม่มีที่ไหนตอบได้

**ตรงกับงานเราไหม** — ตรง และเป็นตัวตั้งต้นของการขึ้นโครงโค้ด เพราะ **หนึ่งโมดูลในเอกสาร เท่ากับหนึ่งไฟล์ในโค้ด**

# 3.5 ภาพรวมสถาปัตยกรรม (C1 และ C2)

**ใส่มาทำไม** — อาจารย์ถามถึง C2 และ C3 โดยตรง และบทที่ 1 ประกาศสถาปัตยกรรมไว้เป็นสมมติฐาน จึงต้องมีที่แสดงว่าทำจริงอย่างไร

**C4 คืออะไร** — เป็นวิธีอธิบายสถาปัตยกรรมด้วยแผนภาพหลายระดับที่ซูมเข้าหากันทีละชั้น

| ระดับ | มองเห็นอะไร | เขียนให้ใครอ่าน |
|---|---|---|
| **C1 System Context** | ระบบเป็นกล่องเดียว ใครใช้ ต่อกับอะไร | เจ้าของร้าน |
| **C2 Container** | หน่วยที่ติดตั้งแยกกัน | ผู้ติดตั้งระบบ |
| **C3 Component** | โมดูลภายในแต่ละหน่วย | ผู้พัฒนา |

**ตัวอย่างให้เห็นภาพ ที่ต้องเล่าให้ได้** — ระบบเป็น **สามชั้นเชิงตรรกะ แต่สี่หน่วยติดตั้ง** ทำไมไม่เท่ากัน

> เพราะ **เครื่องพิมพ์ใบเสร็จเสียบสาย USB อยู่กับเครื่องหน้าร้าน** บริการพิมพ์จึงต้องรันบนเครื่องนั้นเสมอ ส่วนระบบหลังบ้านย้ายไปเซิร์ฟเวอร์ที่ไหนก็ได้ **ถ้ารวมสองส่วนไว้ด้วยกัน ทั้งระบบจะถูกตรึงอยู่กับเครื่องหน้าร้านตลอดไป**

เหตุผลที่แยกจึงไม่ใช่เพราะโค้ดใหญ่ แต่เพราะข้อจำกัดทางกายภาพ

**จุดที่ต้องชี้เองใน C1** — **ลูกค้าอยู่นอกกล่อง ไม่ใช่ผู้ใช้ระบบ** เพราะไม่มีบัญชีและไม่ได้กดอะไรเลย แค่สแกน QR กับรับใบเสร็จ · และ **ไม่มีเส้นไปหาธนาคาร** เพราะเราไม่ต่อ Payment Gateway ระบบจึงไม่รู้เองว่าเงินเข้าแล้วหรือยัง

# 3.6 ส่วนประกอบภายใน (C3)

**ใส่มาทำไม** — เป็นการซูมเข้าไปในกล่อง “ระบบหลังบ้าน” ของ C2 เพื่อแสดงว่าโมดูลทั้งสิบเรียกหากันอย่างไร

**ตัวอย่างให้เห็นภาพ** — ทำไมต้องแยก **ส่วนควบคุมการจ่ายสินค้า (Dispatch Controller)** ออกจาก **บริการตัดสต็อก (Stock Deduction Service)** ทั้งที่ทำงานต่อกัน

> ส่วนควบคุมรับผิดชอบเรื่องรับคำขอ ตรวจสิทธิ์ และแปลงผลเป็นคำตอบ ส่วนบริการตัดสต็อกรับผิดชอบตรรกะธุรกรรมล้วน ๆ
>
> **ประโยชน์ที่ได้คือเวลาทดสอบกรณีสองคนกดพร้อมกัน เราเรียกบริการตัดสต็อกตรง ๆ ได้เลย ไม่ต้องจำลองคำขอผ่านเครือข่ายทั้งเส้นทาง** ซึ่งทำให้ทดสอบ NFR-04 ได้อย่างน่าเชื่อถือ

**ตรงกับงานเราไหม** — ตรง และมีกฎหนึ่งข้อที่ต้องอธิบายได้ คือ **ไฟล์ `app.js` เป็นที่เดียวที่ประกาศเส้นทางและสิทธิ์** ไม่มีโมดูลไหนประกาศเส้นทางของตัวเอง เหตุผลคือถ้าถูกถามว่าเส้นทางไหนใครเข้าได้บ้าง เปิดไฟล์เดียวตอบได้ครบ และทำให้ลืมใส่การตรวจสิทธิ์ไม่ได้

# 3.7 การออกแบบฐานข้อมูล

**ใส่มาทำไม** — เป็นที่ที่ทฤษฎีจากบทที่ 2 ลงมาเป็นของจริงมากที่สุด

**ตัวอย่างให้เห็นภาพ — สี่การตัดสินใจที่ต้องอธิบายได้**

> **หนึ่ง เก็บราคา ณ เวลาที่ขายแยกจากราคาปัจจุบัน** วันที่ 1 ลูกค้าซื้อปูน 150 บาท 10 ถุง เป็นเงิน 1,500 บาท วันที่ 20 ปูนขึ้นเป็น 165 บาท **ถ้าใบเสร็จเก็บแค่รหัสสินค้ากับจำนวน ใบเก่าจะกลายเป็น 1,650 บาททันที** ยอดขายย้อนหลังทั้งเดือนเพี้ยนหมด
>
> **สอง เก็บยอดคงเหลือหลังเปลี่ยนไว้ในบันทึกการเคลื่อนไหวด้วย** ดูเหมือนข้อมูลซ้ำซ้อน แต่เวลานับของแล้วได้ 48 ถุงขณะที่ระบบบอก 50 **ถ้าไม่เก็บ จะรู้แค่ว่าหาย 2 ถุง แต่ไม่รู้ว่าหายตรงรายการไหน**
>
> **สาม ไม่มีสถานะตะกร้าสินค้า และช่องเวลาที่ชำระเงินต้องมีค่าเสมอ** แปลว่าคำสั่งซื้อเข้าฐานข้อมูลก็ต่อเมื่อจ่ายเงินแล้ว ป้องกันกรณีลูกค้าลืมกระเป๋าตังค์แล้วเดินออกไป ทิ้งใบค้างไว้ปนยอดขาย
>
> **สี่ ฐานข้อมูลบังคับว่ายอดต้องไม่ติดลบ** เป็นแนวป้องกันชั้นสุดท้าย ถ้าตรรกะในโปรแกรมพลาด ฐานข้อมูลจะปฏิเสธและยกเลิกธุรกรรม **แทนที่จะปล่อยให้ข้อมูลผิดถูกบันทึกอย่างเงียบ ๆ**

**หัวข้อ 3.7.3 คือหัวใจของโครงงาน** ลำดับเจ็ดขั้นของธุรกรรมยืนยันจ่ายสินค้า จุดที่ต้องอธิบายให้ได้คือ **ขั้นที่ 2 กับขั้นที่ 3 เป็นคนละเรื่องกัน**

| ขั้น | ป้องกันอะไร | ระดับไหน |
|---|---|---|
| **2 เปลี่ยนสถานะแบบมีเงื่อนไข** | จ่ายซ้ำใบเดียวกัน | ระดับคำสั่งซื้อ |
| **3 หักยอดแบบมีเงื่อนไข** | สต็อกติดลบ | ระดับสินค้า |

**การป้องกันอย่างหนึ่งไม่ได้ป้องกันอีกอย่างโดยอัตโนมัติ** จึงต้องมีทั้งคู่ · และขั้นที่ 7 คือการตรวจแจ้งเตือน **อยู่นอกธุรกรรมโดยตั้งใจ** เพราะเป็นงานผลพลอยได้ ถ้าสร้างแจ้งเตือนพลาด ไม่ควรทำให้การจ่ายของที่เกิดขึ้นจริงถูกยกเลิกตาม

# 3.8 แผนภาพลำดับ กิจกรรม และสถานะ

**ใส่มาทำไม** — หัวข้อก่อนหน้าบอกว่าระบบ **ประกอบด้วยอะไร** หัวข้อนี้บอกว่าระบบ **ทำงานอย่างไรเมื่อเวลาผ่านไป** ซึ่งเป็นคนละเรื่อง

**ตัวอย่างให้เห็นภาพ** — แผนภาพกิจกรรมแสดงกระบวนการสองฝั่งคู่ขนาน คือหน้าร้านกับคลัง **จุดที่ระบบเข้าไปเปลี่ยนกระบวนการเดิมอยู่ตรงหลังยืนยันรับชำระเงิน**

> **เดิม** คลังรู้เรื่องตอนลูกค้าเดินมาถึงพร้อมใบเสร็จ แล้วค่อยเริ่มจัดของ **ใหม่** คลังรู้ทันทีที่จ่ายเงินเสร็จ เริ่มจัดของระหว่างลูกค้ากำลังเดินมา

นี่คือจุดเดียวที่ลดเวลารอคอยตามวัตถุประสงค์ข้อ 1.2.2 ถ้าอาจารย์ถามว่าลดเวลาได้ตรงไหน ชี้ตรงนี้

**ส่วนแผนภาพสถานะ** ต้องชี้ให้ได้ว่า **“ส่งมอบสำเร็จ” เป็นปลายทางที่ย้อนไม่ได้ และเกิดได้ทางเดียวคือผ่านธุรกรรมตัดสต็อกที่สำเร็จ** ตั้งค่าตรง ๆ จากหน้าจอจัดการข้อมูลไม่ได้ เพราะถ้าอนุญาต สถานะจะไม่เป็นหลักฐานอีกต่อไปว่าสต็อกถูกหักแล้วจริง

# 3.9 การออกแบบหน้าจอ

**ใส่มาทำไม** — ต้องแสดงว่าหน้าจอออกแบบจากเงื่อนไขการใช้งานจริง ไม่ใช่จากความสวยงาม

**ตัวอย่างให้เห็นภาพ** — หลักสามข้อของเรามีที่มาจากหน้างานทั้งหมด

> **หนึ่งหน้าจอหนึ่งงานหลัก** เพราะพนักงานใช้ตอนมีลูกค้ารออยู่ ตัวเลือกเยอะจะกดผิด **ตัวเลขใหญ่และชิดขวา** เพราะยอดเงินต้องเทียบกันในแนวตั้งได้เร็ว **ยืนยันก่อนการกระทำที่ย้อนไม่ได้** เพราะกดยืนยันจ่ายแล้วสถานะเข้าปลายทางทันที ย้อนไม่ได้

**เรื่องอุปกรณ์เคลื่อนที่** ไม่ได้ทำเพราะสมัยนิยม แต่เพราะ **พนักงานคลังเดินอยู่ในโกดัง ไม่ได้นั่งหน้าคอมพิวเตอร์** ปุ่มยืนยันจึงต้องใหญ่พอกดด้วยนิ้วได้แม่นแม้สวมถุงมือ ส่วนหน้าจอพนักงานขายกับผู้จัดการออกแบบสำหรับจอคอมพิวเตอร์เป็นหลักเพราะใช้ประจำที่

# 3.10 แผนการทดสอบ

**ใส่มาทำไม** — ปิดวงจรของทั้งบท ทุกกรณีทดสอบต้องย้อนกลับไปหาความต้องการที่ตั้งไว้ในหัวข้อ 3.2 ได้

**ตัวอย่างให้เห็นภาพ** — **TC-06 คือกรณีทดสอบที่สำคัญที่สุดของโครงงาน** เพราะพิสูจน์วัตถุประสงค์ข้อ 1.2.3 โดยตรง

> และเป็นกรณีที่ **ทดสอบด้วยมือแทบไม่ได้** เพราะต้องให้สองเครื่องกดชนกันในระดับเสี้ยววินาที นั่งกดเองยังไงก็ไม่ชน เราจึงใช้สคริปต์ยิงคำขอพร้อมกัน ทำให้ทดสอบซ้ำได้และได้ผลเหมือนเดิมทุกครั้ง

**จุดที่ต้องแยกให้ออก** — TC-06 กับ TC-08 ทดสอบคนละกลไก TC-06 ทดสอบการล็อกระดับคำสั่งซื้อ ส่วน TC-08 ทดสอบการล็อกระดับสินค้า ตรงกับสองขั้นในหัวข้อ 3.7.3

**ที่ต้องรู้ตัวไว้** — TC-05, TC-10, TC-11 และ TC-12 ยังไม่มีชุดทดสอบอัตโนมัติ ตรวจด้วยมืออยู่ ถ้าถูกถามให้ตอบตรง ๆ ว่ารู้ตัวและระบุไว้แล้วในเอกสารโครงโค้ด

# ระบบเรามีกี่โมดูล

> **คำตอบสั้น ๆ — 10 โมดูลหลักฝั่งหลังบ้าน · 3 ส่วนประกอบสนับสนุน · 9 หน้าจอฝั่งหน้าเว็บ · ติดตั้งแยกเป็น 4 หน่วย**

**การแบ่งโมดูลไม่ได้แบ่งตามความสะดวก** แต่ได้มาจาก use case โดยใช้หลักว่า **แต่ละโมดูลรับผิดชอบเรื่องเดียว และเปลี่ยนแปลงด้วยเหตุผลเดียว** ตามหัวข้อ 2.1.1 ของบทที่ 2

## สิบโมดูลหลัก และใครใช้

| # | โมดูล | ทำอะไร | ใครใช้ |
|---|---|---|---|
| **1** | ส่วนควบคุมการเข้าสู่ระบบ (Auth Controller) | ตรวจชื่อผู้ใช้กับรหัสผ่าน ออกโทเคน | **ทุกบทบาท** |
| **2** | ส่วนควบคุมข้อมูลสินค้า (Product Controller) | เพิ่ม แก้ไข ค้นหาสินค้า ตั้งจุดสั่งซื้อเพิ่ม | **อ่าน** ทุกบทบาท · **เขียน** ผู้จัดการ |
| **3** | ส่วนควบคุมคำสั่งซื้อ (Order Controller) | คำนวณยอด สร้างเลขที่ บันทึกคำสั่งซื้อ เรียกสร้าง QR | พนักงานขาย, ผู้จัดการ |
| **4** | ส่วนจัดการคิวงาน (Queue Manager) | ให้รายการคิว ปรับสถานะ ผลักเหตุการณ์ | พนักงานคลัง, ผู้จัดการ |
| **5** | ส่วนควบคุมการจ่ายสินค้า (Dispatch Controller) | รับคำขอยืนยันจ่าย คุมลำดับธุรกรรม | พนักงานคลัง, ผู้จัดการ |
| **6** | บริการตัดสต็อก (Stock Deduction Service) | ตัดสต็อกแบบอะตอมมิก กันจ่ายซ้ำ | **ไม่มีผู้ใช้เรียกตรง** โมดูลที่ 5 เรียก |
| **7** | ส่วนรวบรวมข้อมูลแดชบอร์ด (Dashboard Aggregator) | รวบรวมสถิติสองระดับ | **ปฏิบัติการ** ผู้จัดการ, คลัง · **บริหาร** ผู้จัดการ |

| # | โมดูล | ทำอะไร | ใครใช้ |
|---|---|---|---|
| **8** | ส่วนตรวจเงื่อนไขแจ้งเตือน (Alert Engine) | ตรวจจุดสั่งซื้อเพิ่มและคิวค้าง | **อ่าน** ผู้จัดการ, คลัง · **กดอ่านแล้ว** ผู้จัดการ |
| **9** | บริการบันทึกประวัติ (Audit Log Service) | บันทึกเหตุการณ์ ค้นหาย้อนหลัง | **เขียน** ทุกโมดูล · **อ่าน** ผู้จัดการ |
| **10** | ส่วนเชื่อมต่อบริการพิมพ์ (Print Service Client) | ส่งคำสั่งพิมพ์ใบเสร็จ | **ไม่มีผู้ใช้เรียกตรง** โมดูลที่ 3 เรียก |

**คำถามที่ต้องเตรียมตอบ — ทำไมโมดูลที่ 6 กับ 10 ไม่มีใครใช้ แล้วจะแยกทำไม**

> **โมดูลที่ 6** แยกเพราะเป็นตรรกะที่อ่อนไหวที่สุดของระบบ รวมไว้ที่เดียวแล้วเขียนชุดทดสอบเฉพาะได้
> **โมดูลที่ 10** แยกเพราะเป็นส่วนเดียวที่ผูกกับอุปกรณ์จริง จึงต้องย้ายไปอยู่กับเครื่องพิมพ์

## มองกลับด้าน — บทบาทหนึ่งใช้โมดูลอะไรบ้าง

ตารางข้างบนอ่านจากโมดูลไปหาคน ตารางนี้อ่านกลับจากคนไปหาโมดูล **ใช้ตอบเวลาอาจารย์ถามว่า “พนักงานขายใช้อะไรบ้าง”**

| บทบาท | โมดูลที่เรียกได้เอง | หน้าจอที่เข้าได้ |
|---|---|---|
| **พนักงานขาย** | 1 เข้าสู่ระบบ · 2 สินค้า (อ่านอย่างเดียว) · 3 คำสั่งซื้อ | เข้าสู่ระบบ, ขายหน้าร้าน |
| **พนักงานคลังสินค้า** | 1 เข้าสู่ระบบ · 2 สินค้า (อ่าน) · 4 คิวงาน · 5 จ่ายสินค้า · 7 แดชบอร์ดปฏิบัติการ · 8 แจ้งเตือน (อ่าน) | เข้าสู่ระบบ, คิวรอจ่าย, ยืนยันจ่าย, แดชบอร์ดปฏิบัติการ, แจ้งเตือน |
| **ผู้จัดการ** | ทุกโมดูล | ทั้ง 9 หน้าจอ |

**สองเรื่องที่ต้องอธิบายได้**

**หนึ่ง — ทำไมผู้จัดการเข้าได้เกือบทุกอย่าง** เพราะร้านเล็ก วันไหนคนไม่พอผู้จัดการต้องลงมาขายหรือจ่ายของเองได้ นี่เป็นเงื่อนไขจริงของร้าน ไม่ใช่การออกแบบที่หละหลวม

**สอง — เข้าถึงหน้าจอได้ ไม่เท่ากับทำได้ทุกอย่าง** ตัวอย่างชัดที่สุดคือหน้าแจ้งเตือน **พนักงานคลังเปิดอ่านได้ แต่กดทำเครื่องหมายว่าอ่านแล้วไม่ได้** สิทธิ์นั้นเป็นของผู้จัดการ · และหน้าสินค้าเปิดให้ทุกคนอ่านเพราะหน้าจอขายต้องแสดงราคากับยอดคงเหลือ แต่แก้ไขได้เฉพาะผู้จัดการ

# วิธีขึ้นโครงโค้ด ฟรอนต์เอนด์และแบ็กเอนด์

**หลักการเดียวที่ต้องจำ — หนึ่งโมดูลในเอกสาร เท่ากับหนึ่งไฟล์ในโค้ด** เปิดตารางที่ 3.13 แล้วชี้ไฟล์ได้ทันที ไม่ต้องค้นหา

**แบ่งโฟลเดอร์ตามหน่วยติดตั้ง ไม่ใช่ตามชนิดไฟล์** เพราะหน่วยติดตั้งคือสิ่งที่ย้ายไปรันคนละเครื่องได้ แบ่งแบบนี้จึงเห็นทันทีว่าอะไรต้องติดตั้งไปด้วยกัน

```
app/
├── backend/          ระบบหลังบ้าน
├── frontend/         ส่วนติดต่อผู้ใช้
├── print-service/    บริการพิมพ์ (ต้องอยู่กับเครื่องพิมพ์)
└── db/               สคริปต์ฐานข้อมูล
```

## แบ็กเอนด์ (Back End)

```
backend/
├── server.js              เปิดพอร์ตและช่องทางค้าง
├── app.js                 ประกาศเส้นทางและสิทธิ์ทั้งหมด — ที่เดียว
├── middleware/auth.js     ตรวจโทเคนและบทบาท คั่นหน้าทุกเส้นทาง
├── config/db.js           พูลการเชื่อมต่อฐานข้อมูล
├── lib/promptpay.js       สร้างรหัส QR ตาม EMVCo
└── components/            สิบไฟล์ = สิบโมดูลในตารางที่ 3.13
```

**กฎที่ต้องรักษา** — `app.js` เป็นที่เดียวที่ประกาศเส้นทางและสิทธิ์ **ไม่มีโมดูลใน `components/` ประกาศเส้นทางของตัวเอง** ถ้าถูกถามว่าเส้นทางไหนใครเข้าได้ เปิดไฟล์เดียวตอบครบ และลืมใส่การตรวจสิทธิ์ไม่ได้

## ฟรอนต์เอนด์ (Front End)

```
frontend/src/
├── App.jsx                ตารางเส้นทางหน้าจอและการกันสิทธิ์ตามบทบาท
├── auth.jsx               เก็บผู้ใช้ที่ล็อกอินและโทเคน
├── api.js                 รวมการเรียกหลังบ้านไว้ที่เดียว
├── realtime.js            ช่องทางค้างและการสลับไปช่องทางสำรอง
├── components/Layout.jsx  โครงหน้าและเมนูที่กรองตามบทบาท
└── pages/                 เก้าไฟล์ = เก้าหน้าจอในตารางที่ 3.15
```

**กฎที่ต้องรักษา** — หน้าจอใน `pages/` **ไม่เรียกหลังบ้านเอง ต้องผ่าน `api.js` เท่านั้น** เพื่อให้เปลี่ยนเส้นทางหรือเพิ่มการจัดการข้อผิดพลาดแล้วแก้ที่เดียวมีผลทุกหน้าจอ

## ลำดับที่ควรสร้าง ถ้าเริ่มจากศูนย์

ลำดับนี้เลือกมาเพื่อให้ **ทดสอบได้เร็วที่สุด** ไม่ใช่เรียงตามความง่าย

| ลำดับ | สร้างอะไร | ทำไมต้องก่อน |
|---|---|---|
| **1** | ฐานข้อมูลจาก `schema.sql` | ทุกอย่างต้องมีที่เก็บข้อมูลก่อน |
| **2** | `config/db.js` และ `middleware/auth.js` | **ทุกโมดูลพึ่งพาสองตัวนี้** ถ้าไม่มีก่อน จะเขียนโมดูลอื่นไม่ได้เลย |
| **3** | โมดูล 1 + หน้าเข้าสู่ระบบ | ได้ระบบสิทธิ์ก่อน จะทดสอบ TC-01 ได้ตั้งแต่ต้น |
| **4** | โมดูล 2, 3 + หน้าขายหน้าร้าน | **ได้ข้อมูลคำสั่งซื้อจริงไว้ใช้ทดสอบขั้นถัดไป** |
| **5** | โมดูล 4 + หน้าคิว แล้วต่อด้วย 5, 6 + หน้ายืนยันจ่าย | หัวใจของระบบ ต้องมีข้อมูลจากขั้นที่ 4 ก่อน |
| **6** | โมดูล 7–10 + หน้าจอที่เหลือ | เป็นส่วนที่อ่านข้อมูลที่มีอยู่แล้ว ทำทีหลังได้ |

**ลำดับนี้ตรงกับการแบ่งระยะพัฒนาในตารางที่ 3.1** โดยข้อ 3 กับ 4 คือระยะที่ 3 · ข้อ 5 คือระยะที่ 4 · ข้อ 6 คือระยะที่ 5 **ไม่ใช่สองแผนคนละชุด แต่เป็นแผนเดียวกันมองคนละมุม**

# สรุปหน้าเดียว

ถ้าอาจารย์ถามคำถามเดียวว่า “เล่าบทที่ 3 มา” ให้พูดตามนี้

> ผมเริ่มจากปัญหาเก้าข้อที่เก็บจากร้าน แปลงเป็นความต้องการ 9 FR กับ 9 NFR แล้วเป็น use case 11 กรณี
>
> พอได้ use case ครบ ผมถึงรู้ว่าต้องมี **10 โมดูลฝั่งหลังบ้านกับ 9 หน้าจอ** และรู้ว่าโมดูลไหนบทบาทไหนเรียกได้ ทั้งหมดอยู่ในตารางเดียวที่หัวข้อ 3.4
>
> สถาปัตยกรรมเป็นผลลัพธ์ ไม่ใช่จุดเริ่ม ผมได้สามชั้นเชิงตรรกะแต่สี่หน่วยติดตั้ง เพราะบริการพิมพ์ต้องอยู่ติดเครื่องพิมพ์
>
> ส่วนที่ผมภูมิใจที่สุดคือหัวข้อ 3.7.3 ธุรกรรมยืนยันจ่ายสินค้า มีล็อกสองชั้นคนละระดับกัน และการแจ้งเตือนอยู่นอกธุรกรรมโดยตั้งใจ ทดสอบด้วย TC-06 ซึ่งต้องใช้สคริปต์ยิงพร้อมกัน เพราะกดมือให้ชนกันจริงไม่ได้